

Robot Operating System (ROS)

- **Introduction: challenges in robotic software development**
- **Nodes, Topics, Messages, Master, Services, Parameters**

ROS



References:

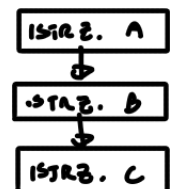
- ROS WIKI - <http://wiki.ros.org/>
- Tutorials - <http://wiki.ros.org/it/ROS/Tutorials>

- To solve the **challenges** of robotic software systems

ROBOT HA TANTE FUNZIONALITÀ
E HA BISOGNO DI QUERSI
SOFTWARE E ALGORITMI

- The **complexity** of robotics software systems

→ **deep stack** starting from driver-level software and continuing up through perception, abstract reasoning, Well beyond the capabilities of any single researcher!



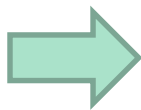
ROBOT PROGRAMMATION

ASINCRONA!

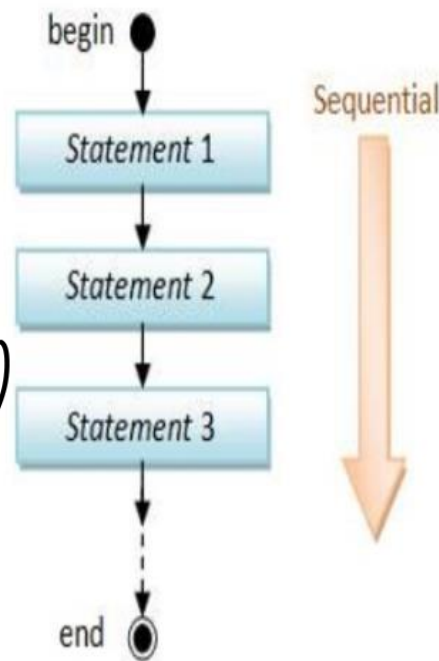
- The world is **asynchronous** → **sequential programming** is ill-suited
- The **diversity** of robotic hardware

→ Different types of robots can have wildly varying hardware, making **code reuse** nontrivial

(CODE PORTABLE E ASSOCIATO
AL HARDWARE)

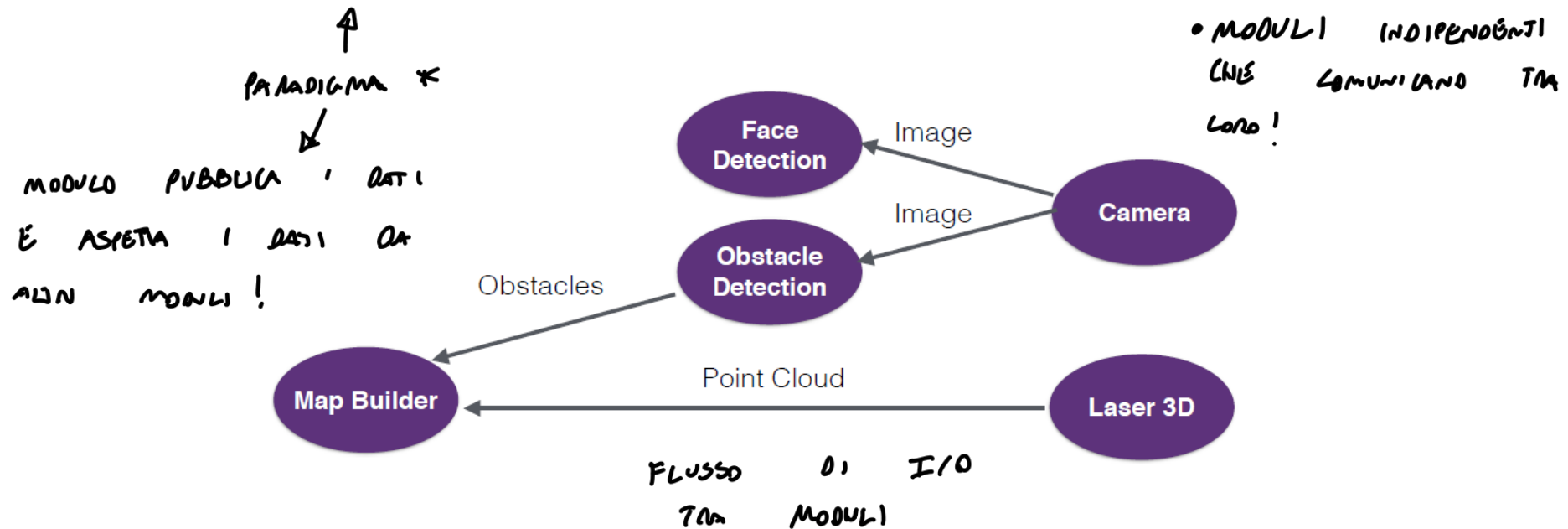


Framework to enable researchers to rapidly develop **new robotic systems** without having to *reinvent the wheel* through use of standards tools and interfaces



ROS NON È UN SISTEMA OPERATIVO
MA UN FRAMEWORK!

- Problem:** The complexity of robotics software systems
- Solution:** Separate Processes → *SCRIVERE I COMITI IN PARTICOLARI PARTI DI SOFTWARE OBBLIGATO!*
 - One process for each functionality
 - Interfaces between processes: **data flows** (*CODE INDIPENDENTE DAGLI ALTRO*)
 - Publish/Subscribe:** each process receives only data it requires



- **Problem:** The world is asynchronous

- Conventional programming: Sequence of instructions

```
goForward(1);  
Image image = camera.getImage();  
double distance = computeDistanceToObject(image);  
goForward(distance - 1);  
(x, y) = getMyPositionFromTheEncoderCounts();
```

→ NON È ADATTO
ALLA PROGRAMMAZIONE
ROBOTICA!

- What happens if an obstacle appears while you are going forward?
- What happens to the encoder data while you are avoiding the obstacle?

- **Solution:** Callbacks (FEEDBACK)

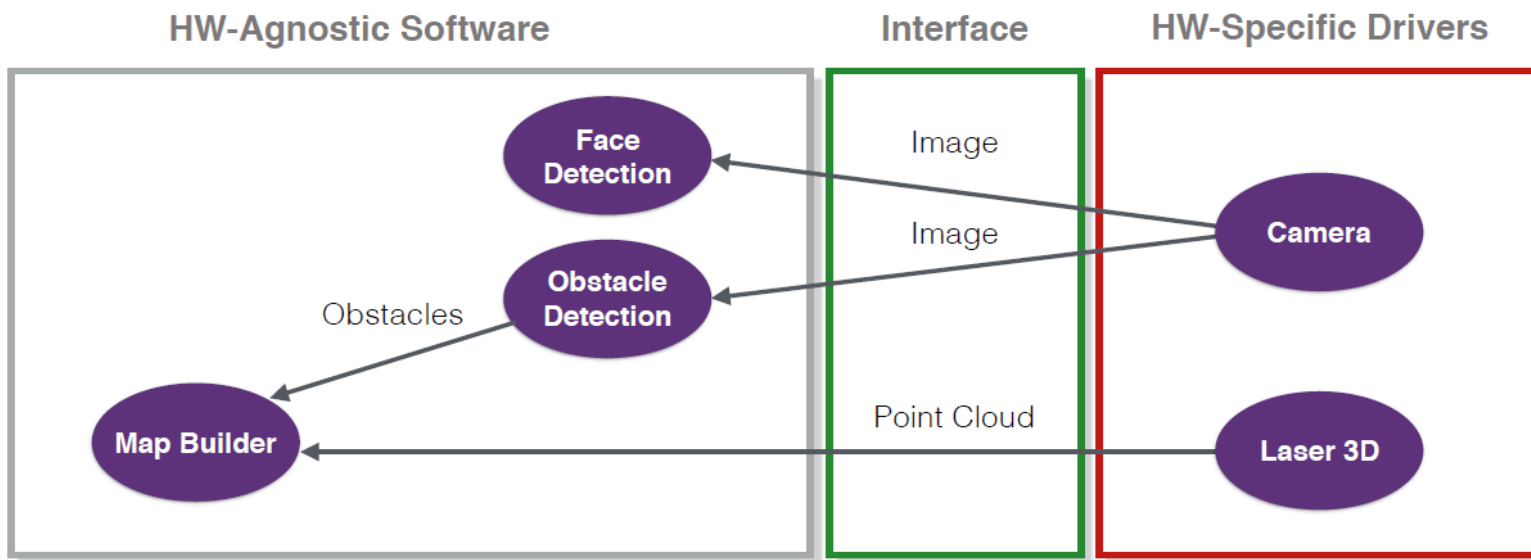
- A **Callback** handles the asynchronous events
 - Function which is called whenever data is available for processing.
 - An asynchronous callback can happen at any time.

→ ARRIVO DI UN CERTO
DATO = D EVENTO
SIGNIFICANTE 4

- **Problem:** The diversity of robotic hardware
- **Solution:** Hardware abstraction

→ **Reusable robotics code**

TANTI HARDWARE ED TANTI CODICI
MULTIUSABILI
↓
INTERFACCIE
E ASTRUZIONI



- PARADIGMA PUBLISH - SUBSCRIBE PER LA GESTIONE DELLA FOTOCAMERA E DELLE IMMAGINI 3D

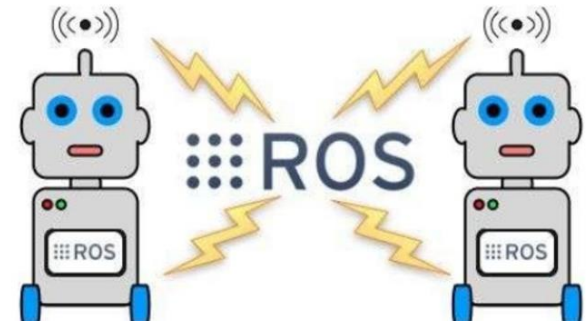
- I MODULI SI ASSELTANO DEI DATI E NE LANCIAO AZIONI!

- Software **framework** for robot software development
- A open source, **meta-operating system** for robots →
 - Services from operating system :

Funzionalità
sistema
ROS

- hardware abstraction
- low-level device control
- implementation of commonly-used functionality
- message-passing between processes
- package management

- It also provides tools and libraries for obtaining, building, writing, and running code across multiple computers (in ROS non esiste un linguaggio preferito)

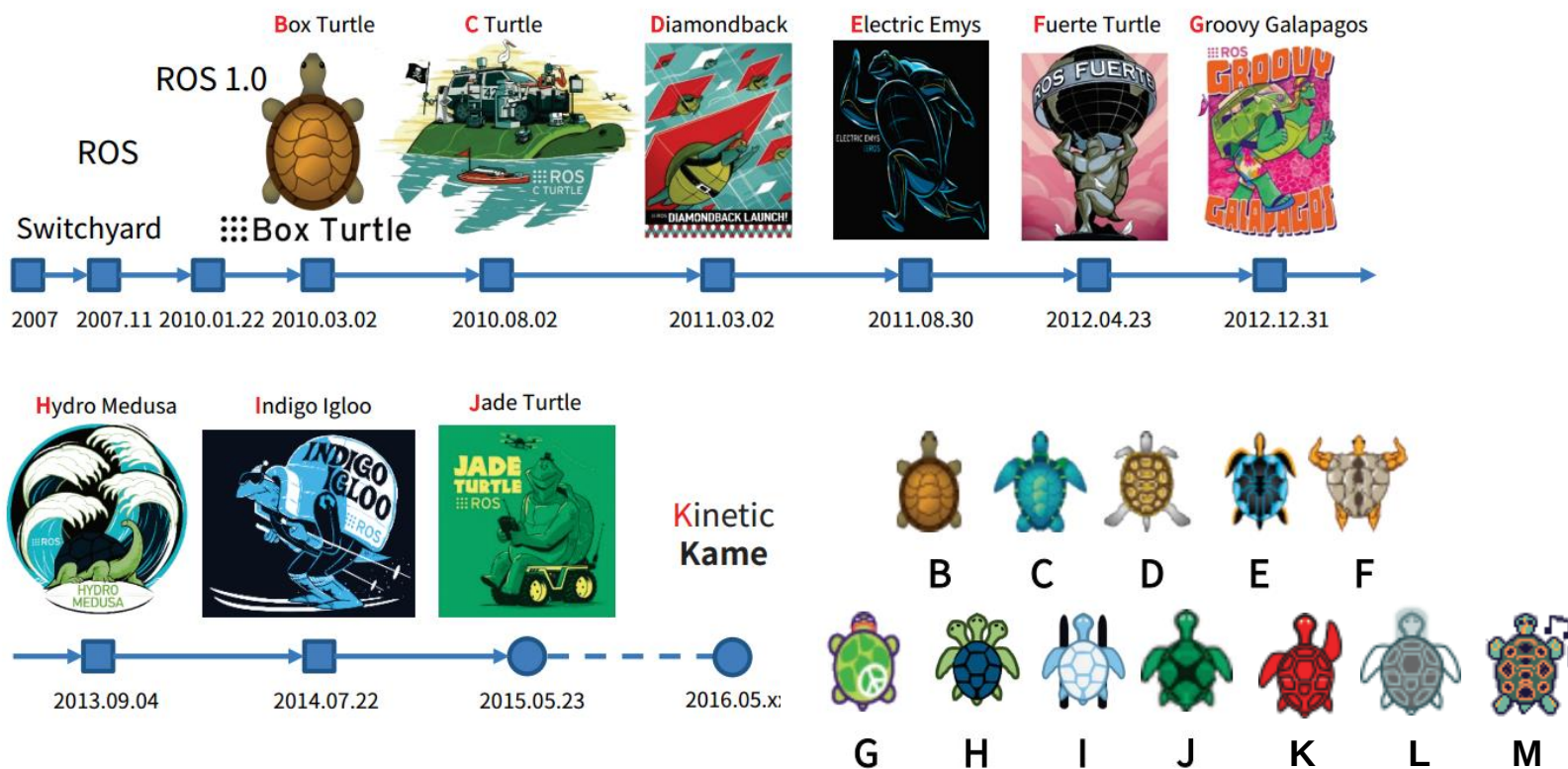


- An **architecture** for inter-process/inter-machine communication and configuration
- A **language-independent** architecture (c++, python, lisp, java, and more)
- A scalable **platform** (from ARM cpus to Intel Xeon clusters)
- Standard packaging and building **environment**
- Development **tools** for system runtime and data analysis
- Open-source under permissive **BSD licenses** (ROS core libraries) → do not require that derivative works based on BSD-licensed software make the source code for such derivative works freely available

What is **NOT** ROS?

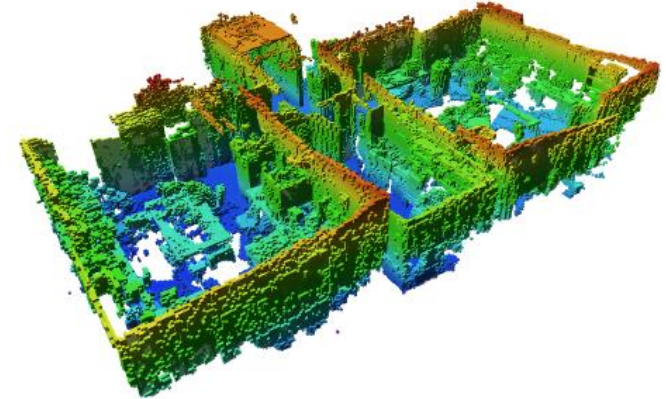
- No confusion please!
- ROS **isn't**:
 - A “actual” operating system
 - A programming language
 - A programming environment / IDE
 - A **real time** architecture → ROS is fast, but it does not provide guarantees about the **timing** of operations

- A ROS distribution is a versioned set of ROS packages (update e versioni)
- These distributions are similar to Linux distributions and provide a set of compatible software for others to use and build upon

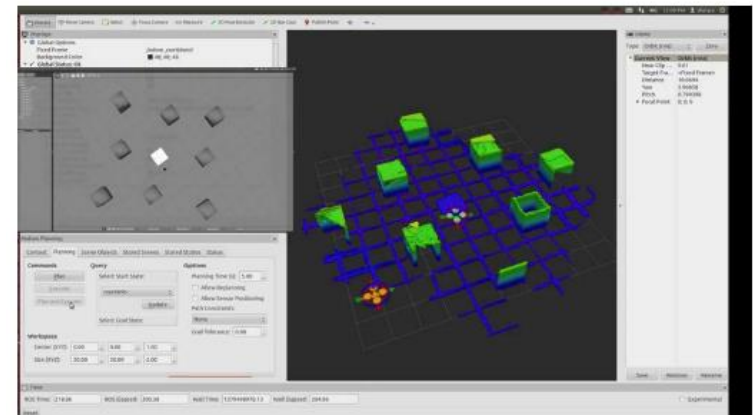


TARSA NELLA MANO IL COMPONENTE
NEL ROBOT NEL PROGRAMMA!

- Perception and Mapping
 - Computer Vision (OpenCV)
 - Stereo Vision / range sensors (PCL)
 - 2D/3D LIDAR (Velodyne integration)
 - Object identification
 - 2D Mapping (mapserver, gmapping)
 - 3D Mapping (octomap)



- Control and Planning
 - Controllers (ROS-Industrial)
 - Motion planning (MoveIt!, OMPL)
 - Navigation stack (2d_nav)
 - Manipulation and grasping



The ROS core is a collection of nodes and programs that are pre-requisites of a ROS-based system:

- ROS Master

- Negotiates communication connections
- The peer-to-peer topology requires some sort of lookup mechanism to allow processes to find each other at runtime. (NODE → MASTER → NODE → NODE → NODE)
- Registers and looks up names for ROS graph resources – name service

NO ROSMASTER → NO NODE → NO + NO → NO !

- ROS Parameter Server

- shared, multi-variate dictionary of configuration parameters and other arbitrary data
- to store and retrieve by key parameters at runtime

+

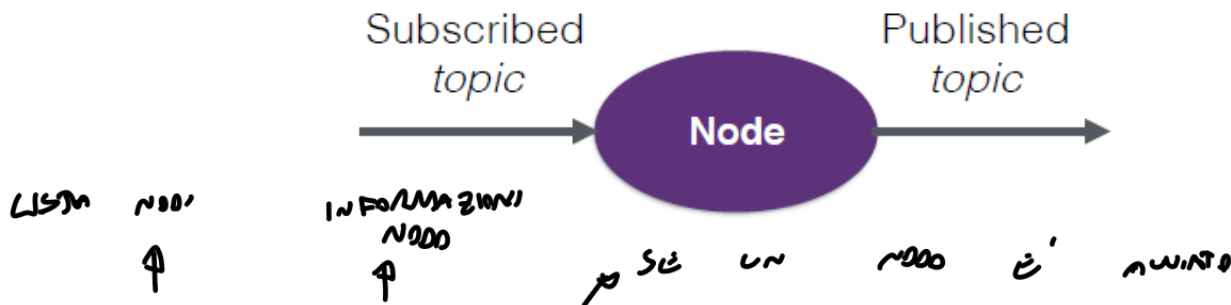
- roscout logging node

- System-wide (inter-process) logging mechanism for human-readable messages (stdout/stderr)

- \$ roscore

- You must have a roscore running in order for ROS nodes to communicate.

- A **node** is an executable file within a ROS package → processes that perform computation
- ROS nodes use a ROS client library to communicate with other nodes
- Nodes can **publish** or **subscribe** to a **Topic** → PUBBLICA DATI SU UN TOPIC
E ASPERA NEL TOPIC ALTRI DATI DA ELABORARE!
- Nodes can also provide or use a **Service**



- `$ rosnode [list / info / ping]`
 - displays information about the ROS nodes
- `$ rosrun PACKAGE EXECUTABLE`
 - Locates PACKAGE and try to find executable named EXECUTABLE in the PACKAGE tree
(PACKAGE = > FILE ESEGUIBILI @ NODI)

- turtlesim → tool made for teaching ROS and ROS packages. (SIMULATORE ROBOT)

Run

→ NODO NEL PACKAGE

- `$ rosrun turtlesim turtlesim_node`

List

- `$ rosnodetree` → LISTA NODI ESEGUITI

Info

- `$ rostopic info /turtle1`

→ INFO NODO

CONSEGNA NODO

Remapping Argument

- `$ rosrun turtlesim turtlesim_node __name:=my_turtle`

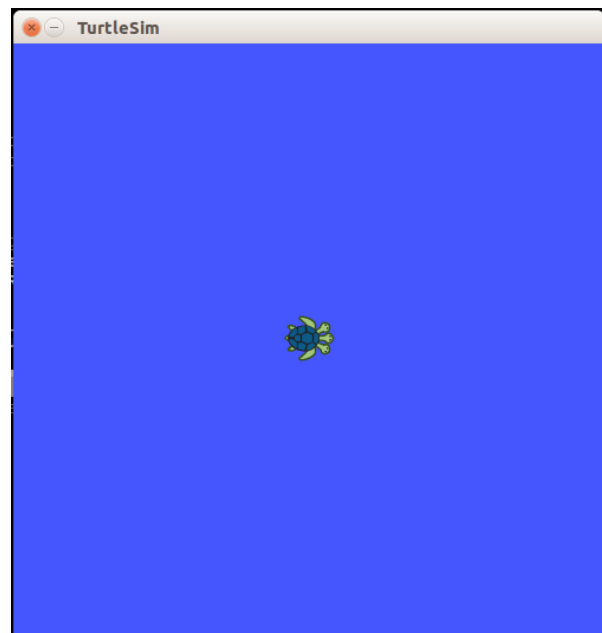
LA RIDEFINIZIONE

Ping

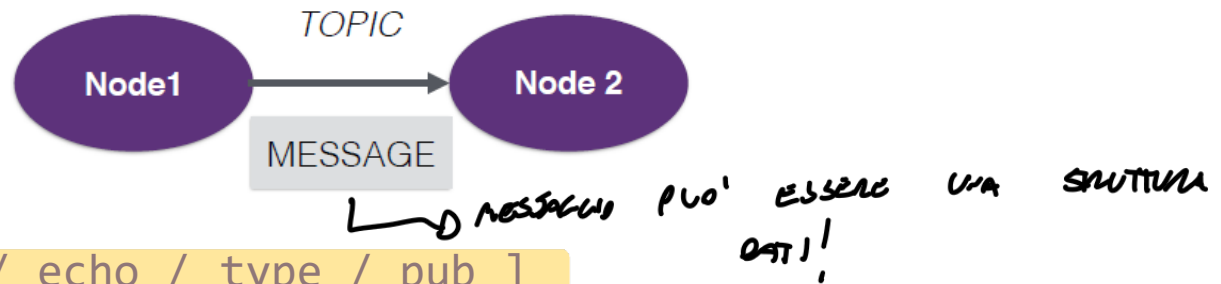
→ VERIFICARE SE IL NODO

- `$ rostopic ping /turtle1`

E' ATTIVO!



- Many-to-many **communication streams** → The idea is to **decouple** the production of information from its consumption
- ROS currently supports TCP/IP-based and UDP-based message transport
- Each **topic** is **strongly typed** by the ROS **message** type used to publish to it. Nodes can only receive messages with a matching type
- A **Message** is a **data structure**, comprising typed fields. Standard primitive types are supported, as are arrays of primitive types and nested structures and arrays



- `$ rostopic [ list / echo / type / pub ]`
 - prints information about active ROS topics
- `$ rosmmsg [show msg]`
 - Displays information about messages, such as displaying the .msg data structures

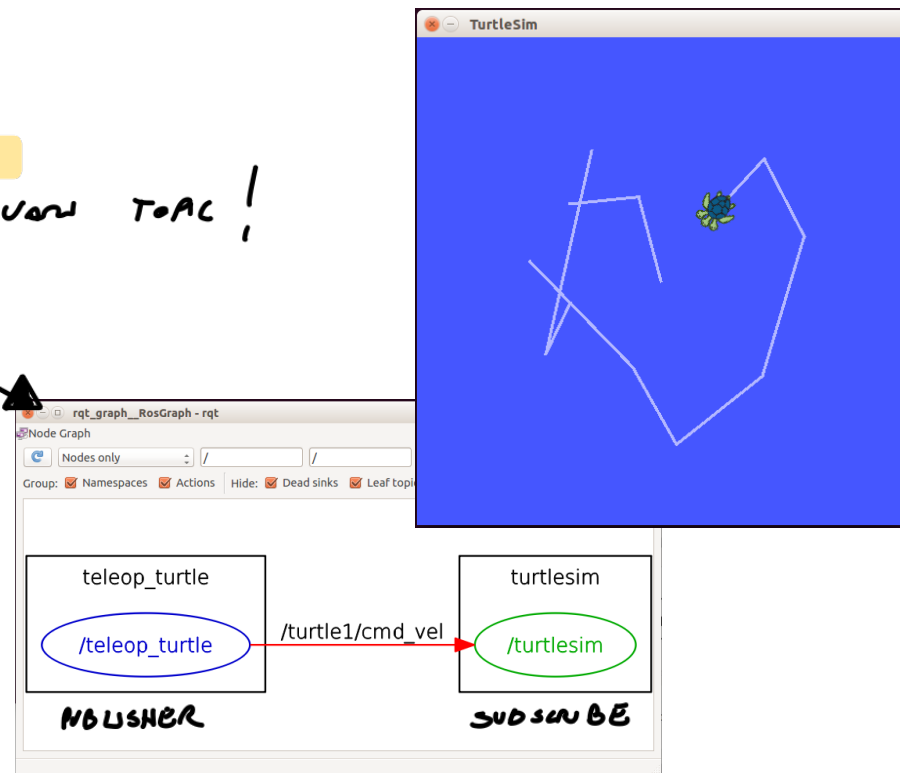
Run

REMOVE LA TARTARUGA !

- `$ rosrund turtlesim turtlesim_node`
- `$ rosrund turtlesim turtle_teleop_key`

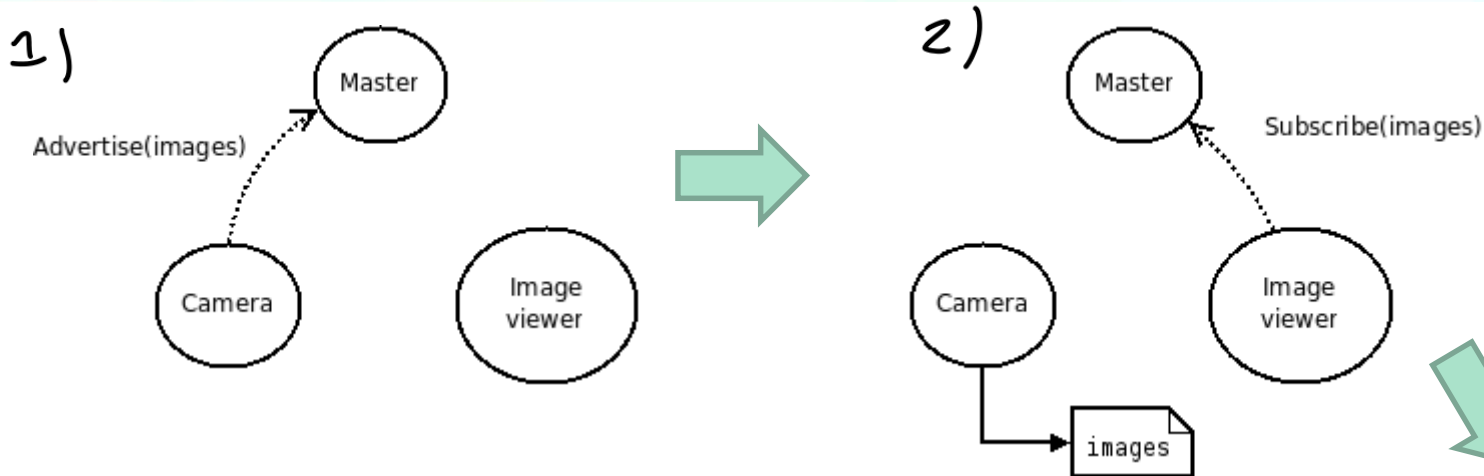
CONNESSIONI TRA I VARI TOPIC !

- `$ rosrund rqt_graph rqt_graph`
- creates a dynamic graph of what's going on in the system

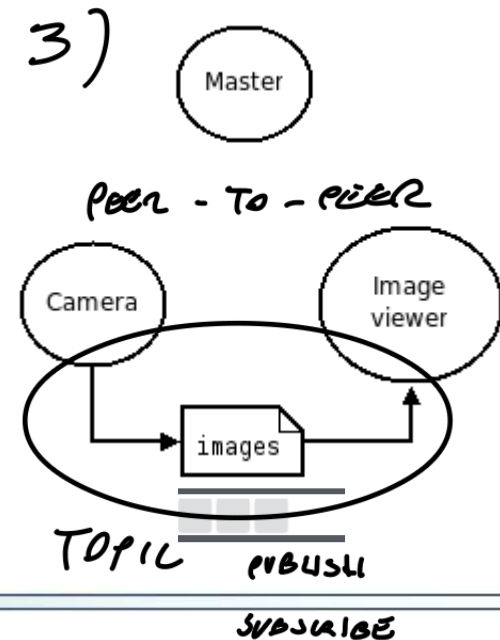


Topic interaction

- `$ rostopic echo /turtle1/cmd_vel`
- `$ rostopic type /turtle1/cmd_vel`
- `$ rosmmsg show geometry_msgs/Twist`
- `$ rostopic pub -l /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'` → single command *LO VELOCITA' ANGLARE TARTARUGA !*
- `$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 0.5]'` → continuous commands

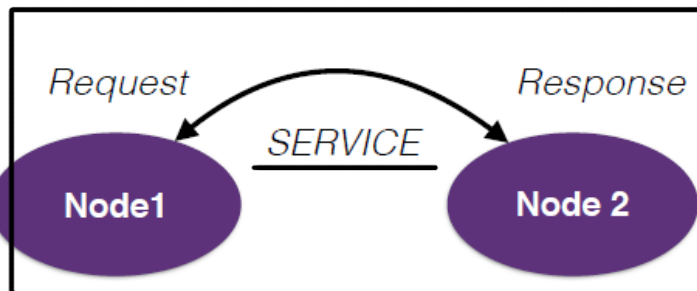


- Camera notifying the master that it wants to **publish images** on the topic **/images**
- Nobody is subscribing to that topic yet so no data is actually sent
- Image_viewer wants to **subscribe** to the topic **/images** to see if there's maybe some images there
- The **topic /images** has both a **publisher** and a **subscriber**
- Once these nodes have located each other they communicate with each other **peer-to-peer**, so they can start **transferring** images to one another



- Services are another way that nodes can communicate with each other → request / reply interactions
- Services allow nodes to send a request and receive a response → are defined by a pair of message structures: one for the request and one for the reply
- A providing ROS node offers a service under a string name, and a client calls the service by sending the request message and awaiting the reply

I NODI RUBBICANO E
RICEVONO MESSAGGI SUI TOPIC
SENZA CONOSCERSI A VICENDA!



UN NODO HA' METTENDO
A DISPOSIZIONE DEI SERVIZI
TRAMITE ROS - TO - PEER

- `$ rosservice [list / call / type]`
 - has many commands that can be used on services
- `$ rossrv [show srv]`
 - Displays information about ROS Service types

↳ STRUTTURA MESSAGGIO RICHIESTA E
RISPOSTA

Service List

- `$ rosservice list`

COMANDI

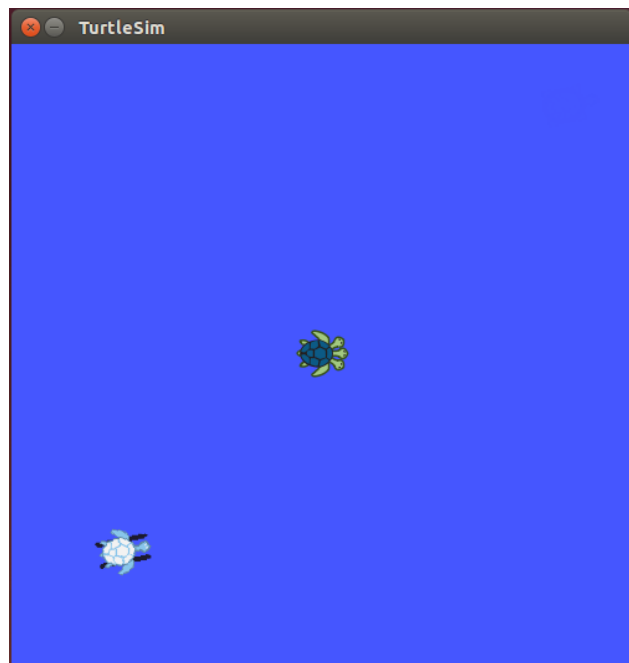
FUNZIONI

Service type

- `$ rosservice type clear` → no args
- `$ rosservice type spawn` → with args
- `$ rossrv show turtlesim/Spawn`

Call Service

- `$ rosservice call /clear`
- `$ rosservice call spawn 2 2 0.2 ""`



PARADIGMA PUBLISH / SUBSCRIBE

ARGOMENTI

PARADIGMA

SERVICES

PRINCIPALI!

→ COPPIO CHIAMA VUOLG

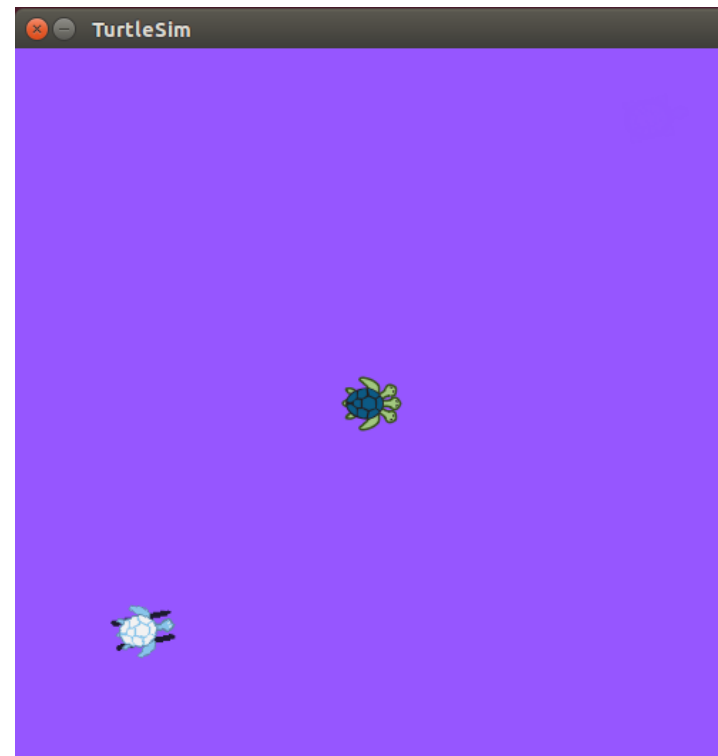
- The **Parameter Server** allows data to be stored by key in a central location. It is currently part of the Master
- **Parameters** are named using the normal ROS naming convention
- ROS parameters have a **hierarchy**
 - to protect parameter names from **colliding**
 - also allows parameters to be accessed **individually** or as a **tree**
- Different data types for parameter values (32-bit integers, Booleans, strings, doubles, lists, ...) and structs
- `$ rosparam [set / get / list]`
 - allows to store and manipulate data on the **ROS Parameter Server**

Parameters

- `$ rosparam list`

Set/Get Parameter

- `$ rosparam set background_r 150`
- `$ rosservice call clear`
- `$ rosparam get background_g`
- `$ rosparam get /`



- Easily **launching** multiple ROS nodes locally and remotely, as well as the machines that they should be run on and setting **parameters** on the Parameter Server
- `$ roslaunch [package] [filename.launch]` *→ più nodi ROS lanciati nello stesso momento!*
 - starts nodes as defined in a **XML .launch file**
 - `$ roscore` is a specialization of the roslaunch tool for bringing up the "core" ROS system *→ processo che gestisce i parametri!*

```
<launch>

<group ns="namespace_1">
  <node pkg="package_name_1" name="node_1_instance_1" type="node_1"/>
</group>

<group ns="namespace_2">
  <node pkg="package_name_1" name="node_1_instance_2" type="node_1"/>
</group>

<node pkg="package_name_2" name="node_2" type="node_2">
  <remap from="input" to="node_2/input"/>
  <remap from="output" to="node_2/output"/>
</node>

<include file="$(find package_name_3)/launch/filelaunch.launch" />

</launch>
```

OUTPUT:

- run ROS core than...

DALLA TURTLESIM

AL

ROBOT



- Bring up turtlesim
- run turtlesim
- teleop the turtle with joystick

vs.

- Bring up the real robot
- run the robot's nodes
- teleop the robot with joystick

